

ЛАБОРАТОРНАЯ РАБОТА №1

Студент	ПИ-23-2 группа	подпись, дата	Жуков М.С. фамилия, инициалы
Студент	ПИ-23-2 группа	подпись, дата	Любанов В.В. фамилия, инициалы
Руководитель	ученая степень, ученое звание	подпись, дата	Первушин О.С. фамилия, инициалы

Липецк 2026

1. Задание кафедры

1. Разработайте оболочку экспертной системы, реализующую прямую цепочку рассуждений, на любом языке программирования.

2. Правила должны формироваться в следующем виде:

ЕСЛИ Объект1=Значение1 ТО Объект2=Значение2

3. Допускается использование логической связки "И". Например,

ЕСЛИ объект1=значение1 И объект2=значение2 ТО объект3=значение3

4. База правил должна быть отделена от программного модуля и представлять собой отдельный текстовый файл.

5. Должна быть реализована возможность редактирования базы правил (изменения, дополнения, удаления правил).

6. В программе должны быть предусмотрены средства для отображения состояния рабочей базы данных.

7. Задайте стартовую ситуацию – начальное состояние рабочей базы данных.

8. Если ни одно из правил не может сработать, программа должна запрашивать у пользователя новые сведения о некотором объекте. Если таких сведений нет, программа завершает работу.

9. Выберите предметную область, которая знакома вам (разработка программного обеспечения, образовательный процесс, игра и т.д.). Составьте базу правил для этой области. Протестируйте работу вашей программы на этой базе правил.

Экспертная система на основе данной стартовой ситуации и своей базы правил должна сделать определенный вывод и отобразить его на экране.

2. Предметная область

В качестве предметной области выбран подбор комплектующих для персонального компьютера. Итоговая сборка определяется несколькими характеристиками, задающимися пользователем независимо друг от друга. Комплектующие в свою очередь связаны между собой ограничениями по совместимости, что позволяет реализовать прямую цепочку рассуждений для оболочки экспертной системы.

3. Алгоритм прямого вывода

Для реализации ядра выбран наивный алгоритм прямого вывода, работающий с тремя сущностями:

- рабочая память — множество фактов вида: Объект=Значение, накопленных в ходе рассуждения, включая стартовую ситуацию;
- база правил — список правил вида: ЕСЛИ Объект1=Значение1 И объект2=значение2 ТО объект3=значение3, каждое из которых может сработать не более одного раза;
- протокол вывода — список сработавших правил вместе с полученными по ним фактами.

Ход вывода:

1. Инициализация. Стартовая ситуация, задаваемая пользователем системы, переносится в рабочую память. Повторяющиеся факты попадают в память единожды.

2. Проверка правил. Просматриваются все правила без отметок о срабатывании. Правило считается применимым, если все его условия истины в текущей рабочей памяти. Сравнение фактов происходит без учёта регистра и краевых пробелов.

3. Срабатывание. Если правило применимо, то оно помечается сработавшим, чтобы не рассматриваться повторно, а его следствие

добавляется в рабочую память как новый факт. Сработавшее правило записывается в протокол вывода.

4. Цикл. П. 2-3 повторяются, пока за очередной полный проход по базе применяется хотя бы одно новое правило. Прямой вывод считается завершённым при окончании прохода без срабатываний.

5. Результат. Формируется итог из стартовых и выведенных фактов, а также протокол вывода.

Запрос дополнительных сведений. Когда вывод останавливается, то система анализирует несработавшие правила и собирает объекты из их условий, которых ещё нет в рабочей памяти, после чего предлагает пользователю обозначить их в стартовой ситуации. Если таких объектов нет, то вывод считается исчерпывающим.

Завершаемость. Алгоритм завершается за конечное число шагов. Каждое правило срабатывает не более одного раза, множество правил конечно, следовательно количество срабатываний также конечно.

4. Структура программы

Программа разделена на три проекта .NET, связанных ссылками и объединённых общим решением ExpertSystem.slnx. Такое разделение позволяет выстроить чёткую иерархию, где ядро не зависит от интерфейса, интерфейс не содержит логики вывода, а тесты проверяют ядро не зависимо от веб-приложения. Также база правил отделена от программного модуля по условиям задания и хранится в файле RuleBase.md.

4.1. ExpertSystem.Core

Является ядром системы, отвечает за модели данных, прямую цепочку вывода и парсер базы правил.

Модели данных:

- Match — факт вида Объект=Значение;
- Rule — правило с одним условием;
- RuleSet — правило с несколькими условиями, соединёнными логическим "И";
- InferenceStep — шаг вывода (сработавшее правило и факт);
- InferenceResult — результат вывода (итоговая рабочая память и протокол вывода).

Логика вывода — статический класс Expert, отвечающий за проверку применимости правил и за реализацию прямой цепочки рассуждений.

Работа с базой — статический класс Parser, реализующий загрузку, сохранение и валидацию JSON при импорте для свода правил.

4.2. ExpertSystem.WebApp

Веб-интерфейс, отвечающий за взаимодействие пользователя с экспертной системой. Содержит страницы свода правил, рассуждения и справки.

4.3. ExpertSystem.Tests

Модульные тесты ядра и парсера на xUnit, проверяющие логику правил, прямую цепочку, подсказки о недостающих объектах и методы парсера.

5. Фрагменты кода

1. Модель факта

```
public class Match
{
    public required string Object { get; set; }
    public required string Value { get; set; }
    public static bool operator ==(Match a, Match b)
    {
        if (a.Object.Trim().ToLower() == b.Object.Trim().ToLower() &&
            a.Value.Trim().ToLower() == b.Value.Trim().ToLower())
            return true;
        return false;
    }
    public static bool operator !=(Match a, Match b) => !(a == b);
    public override bool Equals(object? obj) => obj is Match other && this ==
other;
    public override int GetHashCode() =>
        GetHashCode.Combine(
            Object.Trim().ToLowerInvariant(),
            Value.Trim().ToLowerInvariant());
}
```

2. Модель множественного правила

```
public class RuleSet
{
    public required List<Match> Conditions { get; set; }
    public required Match Consequence { get; set; }
}
```

3. Проверка применимости правила

```
public static bool IsStateMatchesRuleSet(List<Match> states, RuleSet
ruleSet)
```

=> ruleSet.Conditions.All(states.Contains);

6. Взаимодействие с ИИ

При разработке экспертной системы был использован генеративный ИИ-инструмент DeepSeek для помощи в создании базы правил. Для обращения был составлен следующий промпт:

1. Приведи пример правил, которые можно было бы использовать для экспертной системы с использованием прямой цепочки рассуждений по предметной области "подбор комплектующих для ПК"

Полученный ответ:

Вот примерный набор правил для такой ЭС. Я разбил его на несколько слоёв: сначала определяем тип сборки, потом — комплектующие.

Определение сборки:

ЕСЛИ Назначение=Учебный И Бюджет=Низкий ТО Сборка=Базовая

ЕСЛИ Назначение=Игровой И Бюджет=Средний ТО Сборка=Средняя

ЕСЛИ Назначение=Игровой И Бюджет=Высокий ТО Сборка=Мощная

ЕСЛИ Назначение=Монтаж видео И Объём проекта=4K ТО
Сборка=Мощная

Выбор комплектующих:

ЕСЛИ Сборка=Базовая И Платформа=Intel ТО Процессор=Intel Core i3

ЕСЛИ Сборка=Мощная И Платформа=AMD ТО Процессор=AMD Ryzen

7

ЕСЛИ Сборка=Средняя ТО Объём ОЗУ=16 ГБ

ЕСЛИ Сокет=AM5 ТО Тип памяти=DDR5

После полученного ответа была спроектирована собственная база правил, состоящая из 19 правил, которых, как было выявлено, оказалось недостаточно для покрытия нашей системы. После чего был составлен второй промпт:

2. Доработай базу правил на основе текущей, достаточную для покрытия системы. Список текущих правил:

ЕСЛИ Назначение=Офисный ТО Класс сборки=Базовый

ЕСЛИ Назначение=Игровой И Бюджет=Средний ТО Класс сборки=Высокий

ЕСЛИ Назначение=Игровой И Бюджет=Высокий ТО Класс сборки=Максимальный

...

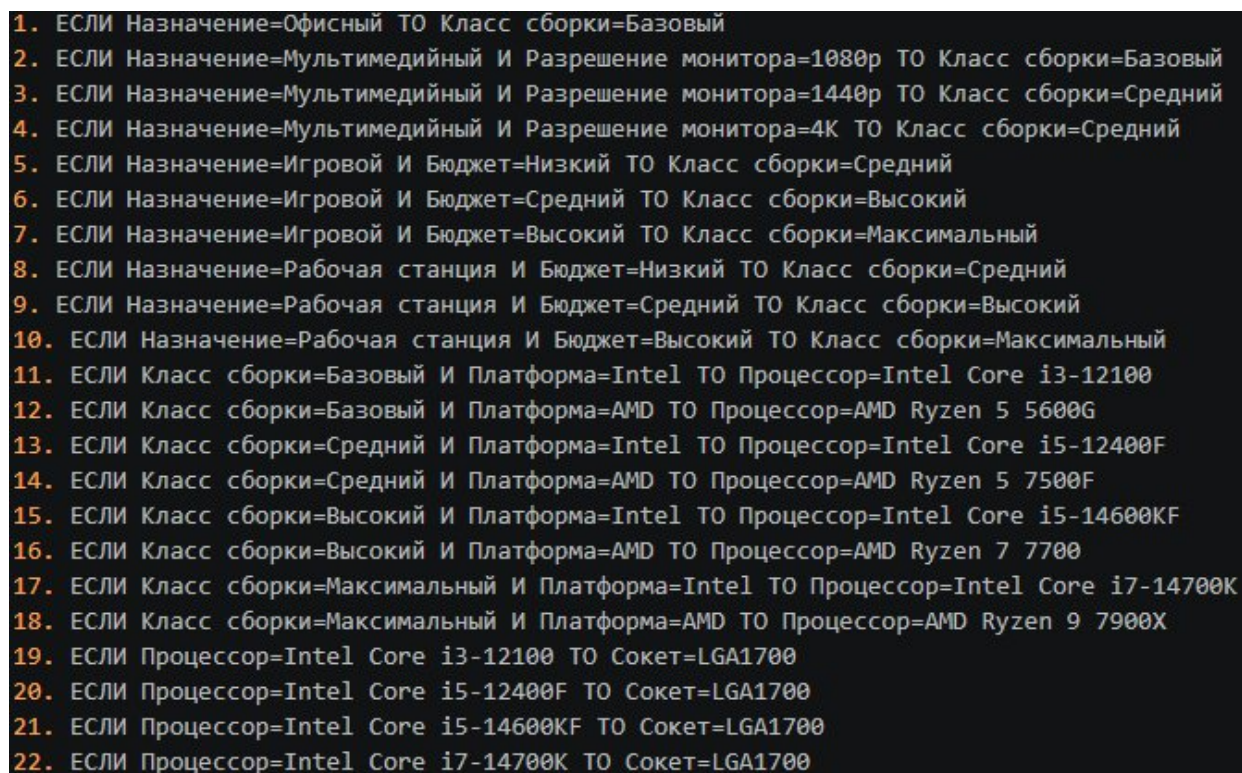
ЕСЛИ Видеокарта=Nvidia GeForce RTX 4070 Super ТО Блок питания=650 Вт

ЕСЛИ Форм-фактор=Стандартный ТО Корпус=ATX Mid-Tower

В качестве ответа была получена база правил, состоящая из 80 правил. После её редактирования была получена окончательная база правил, состоящая из 71 правила.

7. Пример работы

На рисунке 1 представлена часть базы правил.



1. ЕСЛИ Назначение=Офисный ТО Класс сборки=Базовый
2. ЕСЛИ Назначение=Мультимедийный И Разрешение монитора=1080p ТО Класс сборки=Базовый
3. ЕСЛИ Назначение=Мультимедийный И Разрешение монитора=1440p ТО Класс сборки=Средний
4. ЕСЛИ Назначение=Мультимедийный И Разрешение монитора=4K ТО Класс сборки=Средний
5. ЕСЛИ Назначение=Игровой И Бюджет=Низкий ТО Класс сборки=Средний
6. ЕСЛИ Назначение=Игровой И Бюджет=Средний ТО Класс сборки=Высокий
7. ЕСЛИ Назначение=Игровой И Бюджет=Высокий ТО Класс сборки=Максимальный
8. ЕСЛИ Назначение=Рабочая станция И Бюджет=Низкий ТО Класс сборки=Средний
9. ЕСЛИ Назначение=Рабочая станция И Бюджет=Средний ТО Класс сборки=Высокий
10. ЕСЛИ Назначение=Рабочая станция И Бюджет=Высокий ТО Класс сборки=Максимальный
11. ЕСЛИ Класс сборки=Базовый И Платформа=Intel ТО Процессор=Intel Core i3-12100
12. ЕСЛИ Класс сборки=Базовый И Платформа=AMD ТО Процессор=AMD Ryzen 5 5600G
13. ЕСЛИ Класс сборки=Средний И Платформа=Intel ТО Процессор=Intel Core i5-12400F
14. ЕСЛИ Класс сборки=Средний И Платформа=AMD ТО Процессор=AMD Ryzen 5 7500F
15. ЕСЛИ Класс сборки=Высокий И Платформа=Intel ТО Процессор=Intel Core i5-14600KF
16. ЕСЛИ Класс сборки=Высокий И Платформа=AMD ТО Процессор=AMD Ryzen 7 7700
17. ЕСЛИ Класс сборки=Максимальный И Платформа=Intel ТО Процессор=Intel Core i7-14700K
18. ЕСЛИ Класс сборки=Максимальный И Платформа=AMD ТО Процессор=AMD Ryzen 9 7900X
19. ЕСЛИ Процессор=Intel Core i3-12100 ТО Сокет=LGA1700
20. ЕСЛИ Процессор=Intel Core i5-12400F ТО Сокет=LGA1700
21. ЕСЛИ Процессор=Intel Core i5-14600KF ТО Сокет=LGA1700
22. ЕСЛИ Процессор=Intel Core i7-14700K ТО Сокет=LGA1700

Рисунок 1 – Содержимое базы данных правил

На рисунке 2 представлена стартовая ситуация и полученный вывод. В качестве стартовой ситуации заданы факты: Бюджет=Низкий, Назначение=Игровой, Разрешение монитора=1080p, Форм-фактор=Компактный, Платформа=Intel.

Запуск рассуждений

Стартовая ситуация

Известные факты в виде «Объект = Значение».

Бюджет	=	Низкий	✕
Назначение	=	Игровой	✕
Разрешение монитора	=	1080p	✕
Форм-фактор	=	Компактный	✕
Платформа	=	Intel	✕

+ факт

Запустить размышление

Вывод:

- Класс сборки = Средний
- Процессор = Intel Core i5-12400F
- Сокет = LGA1700
- Тип памяти = DDR4
- Объём ОЗУ = 16 Гб
- Материнская плата = Intel B660 (DDR4), Mini-ITX
- Видеокарта = Nvidia GeForce RTX 4060
- Блок питания = 550 Вт
- Охлаждение = Башенный кулер
- Накопитель = NVMe SSD 1 Тб
- Корпус = Mini-ITX

Рисунок 2 – Стартовая ситуация и вывод

На рисунке 3 представлена рабочая база данных и ход рассуждений.

Рабочая база данных			
#	Объект	Значение	Источник
1	Бюджет	Низкий	стартовые данные
2	Назначение	Игровой	стартовые данные
3	Разрешение монитора	1080p	стартовые данные
4	Форм-фактор	Компактный	стартовые данные
5	Платформа	Intel	стартовые данные
6	Класс сборки	Средний	правило #5
7	Процессор	Intel Core i5-12400F	правило #13
8	Сокет	LGA1700	правило #20
9	Тип памяти	DDR4	правило #30
10	Объём ОЗУ	16 Гб	правило #34
11	Материнская плата	Intel B660 (DDR4), Mini-ITX	правило #38
12	Видеокарта	Nvidia GeForce RTX 4060	правило #46
13	Блок питания	550 Вт	правило #56
14	Охлаждение	Башенный кулер	правило #63
15	Накопитель	NVMe SSD 1 Тб	правило #67
16	Корпус	Mini-ITX	правило #71
Ход рассуждений			
1. Правило #5: ЕСЛИ Назначение = Игровой И Бюджет = Низкий ТО Класс сборки = Средний			
2. Правило #13: ЕСЛИ Класс сборки = Средний И Платформа = Intel ТО Процессор = Intel Core i5-12400F			
3. Правило #20: ЕСЛИ Процессор = Intel Core i5-12400F ТО Сокет = LGA1700			
4. Правило #30: ЕСЛИ Сокет = LGA1700 И Класс сборки = Средний ТО Тип памяти = DDR4			
5. Правило #34: ЕСЛИ Класс сборки = Средний ТО Объём ОЗУ = 16 Гб			
6. Правило #38: ЕСЛИ Сокет = LGA1700 И Тип памяти = DDR4 И Форм-фактор = Компактный ТО Материнская плата = Intel B660 (DDR4), Mini-ITX			
7. Правило #46: ЕСЛИ Класс сборки = Средний И Разрешение монитора = 1080p ТО Видеокарта = Nvidia GeForce RTX 4060			
8. Правило #56: ЕСЛИ Видеокарта = Nvidia GeForce RTX 4060 ТО Блок питания = 550 Вт			
9. Правило #63: ЕСЛИ Класс сборки = Средний ТО Охлаждение = Башенный кулер			
10. Правило #67: ЕСЛИ Класс сборки = Средний ТО Накопитель = NVMe SSD 1 Тб			
11. Правило #71: ЕСЛИ Форм-фактор = Компактный ТО Корпус = Mini-ITX			

Рисунок 3 – Рабочая база данных и ход рассуждений.

Выводы

В процессе работы была разработана оболочка экспертной системы, реализующая прямую цепочку рассуждений, а также изучены методы построения экспертных систем, принципы прямой цепочки рассуждений, разобран наивный алгоритм прямого вывода.